

RBefunge ISA

RBefunge is a reversible computation language using direction/momentum and mirrors for control flow. Inspired by Befunge, but fully reversible.

State

- `tape[0..N-1]` — memory (program + data), 8-bit cells
- `IP` — instruction pointer (position)
- `Dir` — direction/momentum (+1 = right, -1 = left)
- `H0` — data head 0 (primary, where +/- act, condition for mirrors)
- `H1` — data head 1 (secondary, for swaps and reversible copy)

All pointers wrap modulo N (tape size). After each instruction, IP advances by `Dir`.

Instructions

Op	Name	Semantics	Inverse
+	increment	<code>tape[H0] += 1</code>	-
-	decrement	<code>tape[H0] -= 1</code>	+
<	H0 left	<code>H0 -= 1</code>	>
>	H0 right	<code>H0 += 1</code>	<
{	H1 left	<code>H1 -= 1</code>	}
}	H1 right	<code>H1 += 1</code>	{
.	copy to H1	<code>tape[H1] += tape[H0]</code>	,
,	copy to H0	<code>tape[H0] += tape[H1]</code>	.
x	swap	<code>swap(tape[H0], tape[H1])</code>	self
[	zero mirror	<code>if tape[H0] == 0: Dir *= -1</code>	self
]	nonzero mirror	<code>if tape[H0] != 0: Dir *= -1</code>	self

All operations are reversible. Mirrors are self-inverse.

True Reversibility

Every RBefunge computation can be run backwards without a history stack:

1. Move IP backwards: `IP = (IP - Dir) mod N`
2. Look at the instruction at the previous position
3. If it's a mirror that would have fired, flip `Dir`
4. Apply the inverse operation

The `r` command in the simulator implements this.

Control Flow with Mirrors

Mirrors conditionally reverse the direction of execution:

- [(zero mirror): bounces when `tape[H0] == 0`, passes otherwise
-] (nonzero mirror): bounces when `tape[H0] != 0`, passes otherwise

Loop Pattern: `] body ]`

For a “while nonzero” loop, use nonzero mirrors on both sides:

`] - ]`

With `tape[H0] = 3`, starting at the `-` with `Dir = +1`:

Step 0: IP=1 (`-`), `Dir=→`, `tape[H0]=3` → decrement to 2
Step 1: IP=2 (`]`), `Dir=→`, `tape[H0]=2` → bounce (2 0), `Dir=←`
Step 2: IP=1 (`-`), `Dir=←`, `tape[H0]=2` → decrement to 1
Step 3: IP=0 (`]`), `Dir=←`, `tape[H0]=1` → bounce (1 0), `Dir=→`
Step 4: IP=1 (`-`), `Dir=→`, `tape[H0]=1` → decrement to 0
Step 5: IP=2 (`]`), `Dir=→`, `tape[H0]=0` → pass (0=0), exit right

Total decrements: 3

Key Insight

Start the loop **inside** the mirrors with a known direction. The mirrors keep bouncing the IP back and forth until the exit condition is met.

Reversible Copy

The `.` and `,` operations implement reversible copy via addition:

```
tape[H1] = 0      # Start with H1 target clear
.                # tape[H1] += tape[H0], now tape[H1] = tape[H0]
```

To uncopy (restore `tape[H1]` to 0):

```
,                # tape[H0] += tape[H1], equivalent to restoring
```

Actually, the inverse of `.` is `,`, which does subtraction during reversal.

Comparison to F***brain

Feature	F***brain	RBefunge
Control flow	J (swap IP with memory)	Mirrors (flip direction)
Direction	IP always +1	IP moves by Dir (± 1)
Garbage cells	Required for loops	Not required
J reversibility	Requires heuristics	N/A
Mirror reversibility	N/A	Always correct

RBefunge's mirrors are **unconditionally reversible** - no ambiguity about where IP came from. The direction (Dir) encodes the execution history.

Example: Countdown with History

```
] - . } ]
```

This decrements `tape[H0]` and copies each value to `tape[H1]`, building a history. The garbage accumulates in `tape[H1]`.